Review of DSC 102
Spring 2026
Profs. Umesh Bellur and Alexander Kolovos

Computer Organization

Foundations & Models

Stored-Program Concept

- What bottleneck did the stored-program idea solve vs. the 1945 hardwired approach (ENIAC)?
- Define hardware vs. software, with one example of each.
- Two identical bit sequences can be data or an instruction — what determines the interpretation?
- What does the Harvard architecture try to solve, and how does it differ from Von Neumann?

Von Neumann Model

- State the two key properties of the Von Neumann (stored-program) model.
- What is the role of the program counter, and how does it normally advance?
- What is the significance of program order in a Von Neumann program?

Dataflow Model

- How does dataflow decide when an instruction executes, vs. Von Neumann?
- Why is the dataflow model inherently more parallel?

Abstraction Layers

- Define abstraction and explain how it improves productivity.
- Why doesn't a high-level programmer normally need to know the underlying ISA?

Hardware, ISA & Execution

Hardware Components

- Name the four key parts of computer hardware and the role of each.
- Match to function: ALU, Control Unit, Program Counter, Registers.
- Why are registers the fastest memory in a computer?
- Contrast DRAM and secondary storage: speed, persistence, cost/capacity.

ISA & Instruction Execution

- What is an ISA, and why is it the 'bridge' between hardware and software?
- Walk through the instruction cycle: fetch → decode → execute → write-back.

Processor Performance

- Why does clock rate (GHz) alone fail to tell you work done per program?
    - Thought exercise: why might 3.6 GHz not beat 2.8 GHz for big-data work?
- What stalls a processor, making actual runtime far higher than clock-rate suggests?
- Why is optimizing main-memory access and cache use critical to performance?

Number Representations

Foundations, Binary & Numbers

Data, Code & Stored-Program

- Difference between data and code — how can one bit sequence be either?
- What determines if a bit pattern is fetched as an instruction vs. read as data?
- What kind of software is TensorFlow? Linux?
- Independent matrix-mults: Control-Flow or Data-Flow accelerator, and why?

Why Binary? Analog to Digital

- Why binary rather than decimal or a higher base?
- Why does analog accumulate error; how does a finite symbol set reduce it?
- Why is unary impractical? Cost of addition (O(N) vs O(log N))?

Bits, Bytes & Bases

- What is a byte, and why is it the basic unit of data types?
- How many integers in 5 bits? How many bits for 5 items?
- Hex of 20 (decimal)? Why is hex a convenient stand-in for binary?

Characters, Strings & Encodings

- What is an encoding, and how does it differ from a data type?
- Why was ASCII inadequate worldwide? Give a failure example.
- What does Unicode solve; how does it stay ASCII-compatible?
- Is code a string? Is a string code?

Numeric Representation

Integers: Signed & Two's Complement

- A Boolean needs 1 bit but takes 1 byte — why are 7 bits "wasted"?
- Why does unsigned store values twice as large as signed of equal width?
- In two's complement, how is a negative formed? Which bit signals it?
- Why one more negative value than positive, and a single zero?
- What is overflow; how does the carry bit let software react to it?

Floating Point (IEEE-754)

- Why is a float standard needed; why must a data scientist know formats?
- Name the three fields (sign, exponent, mantissa) and what each controls.
- What is the biased exponent (E - 127), and what does the bias make easier?
- What is the hidden / implicit bit — why "a free bit of precision"?
- Why is floating-point addition not associative (0.1 + 0.2)?

Precision, Performance & Synthesis

Precision, Byte Order & Serialization

- What does “lower precision” mean for a DL weight; what is gained?
- Self-driving model: Float64 or Float16 if 16 is faster but errs 0.1% more?
- Big- vs little-endian for 0x12345678 — why expose byte order to the programmer?
- Why is serialization needed; the two ways to serialize a number?

CPU Performance

- Define average CPI; how to get total cycles and execution time from a mix?
- Why can memory “dominate” time even when most instructions are fast?
- Speedup when a cache cuts a stall from 100 to 20 cycles?
- 4-byte aligned instrs from 0x1000: next addresses; fraction of valid addresses?

Synthesis & Open-Ended

- How do feature vectors, NN weights, graphs, timestamps, and embeddings all reduce to basic data types?
- How does weight quantization affect a phone's battery running an LLM — at what cost?
- Trace the pipeline: analog voltage to a reliable bit to a float a model can train on.

Memory Hierarchy, Caching and Pipelining

Processors & Memory Hierarchy

Processors & Performance

- What is a load-store architecture, and what role do registers play in it?
- What does the ISA specify, and how does a PL program become machine code?
- Why is it usually impossible for data programs to reach 100% CPU utilization?

Memory/Storage Hierarchy

- Why do CPUs have both registers and caches?
- Order by cost and by access speed: CPU cache, DRAM, flash, magnetic HDD — which is costliest? slowest?
- Roughly how do capacity, price, and access speed trade off across the hierarchy?

Locality of Reference

- Define temporal vs. spatial locality; give a simple program exhibiting each.
- How do caches exploit temporal locality? How do they exploit spatial locality (cache lines)?
- Define caching, prefetching, hit vs. miss/fault, and a cache replacement policy.

Data Layout & Performance

Data Layout & Access Patterns

- Define data layout, data access pattern, and hardware efficiency.
- Why does data layout matter for a program's hardware efficiency?
- What is more important to optimize: data layout or data access pattern?

Exploiting Locality (Matrix Multiply)

- In row-major layout, why does the i-j-k loop order cause B[k][j] misses and stalls?
- Why is the rewritten i-k-j order far more hardware-efficient, despite identical math?
- Why use NumPy instead of writing cache-aware code yourself?

Caching Math & Pipelining

AMAT — Two-Level

- Cache=10ns, DRAM=1.5µs, hit rate=0.7 — compute AMAT.
- If hit rate rises to 0.9, what is the new AMAT and the speedup?

AMAT — Multi-Level

- L1=10ns (0.7), L2=70ns (0.9 on L1 miss), DRAM=1.5µs — compute overall AMAT.
- What fraction of accesses reach DRAM? Recompute AMAT if L1 hit rate is 0.8.

Cache Lines & Access Order

- 64-byte line, 4-byte ints, row-major: how many elements per line; what fraction are hits for row-wise access?
- How do cache misses change for column-wise (j-i) access of the same array?

Pipelining the Processor

- Name the five pipeline stages (IF, ID, EX, MEM, WB) and what each does.
- Does pipelining improve latency or throughput? What limits the pipeline rate?
- What causes pipeline stalls and flushes, and how does branch prediction help?

CPU & Memory Virtualization

CPU Virtualization & Scheduling

- What is a process, and how does it differ from a program? List three concrete differences.
- Describe the three states of a process (Running, Ready, Blocked) and the events that cause transitions between them.
- What information does the kernel store in a process descriptor / PCB, and why is each item needed for a context switch?
- Explain Limited Direct Execution. Why can't the OS simply let a user program run directly on the CPU with no restrictions?
- What is the difference between user mode and kernel mode? Give three operations a process should not be allowed to perform in user mode.
- How does a system call work? Trace the steps from the application request to the return of control to user space.
- Once the OS hands the CPU to a process, how does it ever get control back? Explain the roles of timer interrupts and context switches.
- Walk through what os.fork() does. In the code that prints A, forks, prints B, conditionally waits, then prints C, what output orderings are possible?
- Define arrival time, job length, start time, completion time, response time, and turnaround time.

Scheduling Policies & Concurrency

- Compare FCFS, SJF, SRTF (SCTF), and Round Robin on preemption, need for prior knowledge of job lengths, and effect on average turnaround vs. response time.
- What is the Convoy Effect, and which scheduling policy is most vulnerable to it?
- SJF can produce starvation. Explain how, and describe a scenario where the long job never runs.
- Explain the fundamental tradeoff: why can no preemption-free, priority-free scheduler simultaneously guarantee optimal average waiting time AND bounded waiting (no starvation)?
- Contrast processes and threads on memory space, communication cost, context-switch cost, and crash isolation.
- Show how two threads incrementing a shared counter can produce a lost update (race condition). What is the sequence of interleaved reads and writes?
- How does a lock fix the race condition? Walk through the acquire / blocked / release sequence.
- What is Python's GIL, and how does it affect CPU-bound vs. I/O-bound multithreaded workloads?
- In multiprocessor scheduling, contrast SQMS and MQMS. What is load balancing and why does cache affinity complicate it?

Memory Virtualization

- Why do we need memory virtualization? State the capacity problem and the isolation problem.
- List the three goals of memory virtualization (transparency, efficiency, protection/isolation) and explain what each means.
- What is an address space, and what determines its maximum size?
- Distinguish a virtual (logical) address from a physical address. What role does the MMU play, and why is it implemented in hardware?
- Define external fragmentation and internal fragmentation. Give a cause of each from the slides.
- How does fixed-size paging eliminate external fragmentation? What residual fragmentation remains?
- A virtual address splits into a page number and an offset. Explain how the page table translates this into a physical frame address.
- What problem does the TLB solve, and what happens on a TLB miss?
- What is a page fault? Trace the steps the OS takes to handle one when the page is on disk (swap space).
- When no free frame exists, the OS must pick a victim page. Compare FIFO, OPT, and LRU. Why is OPT not implementable in practice?
- What is thrashing, and how does the Working Set model help an OS decide how many frames to give a process?

Persistence, File Systems

Disks & I/O

- Name three forms of persistent storage devices. Which one has a key latency dichotomy for random vs. sequential access, and why?
- What are the three components of disk I/O time? Which dominate for small, random I/Os?
- For a 7,200 RPM drive, how do you compute the average rotational latency?
- Why can sequential access beat random access by a large margin on a magnetic disk?
- Define IOPS, bandwidth/throughput, and latency. How are IOPS and bandwidth related?
- How do Flash SSDs and NVRAM differ from magnetic disks in addressability, latency, and locality of reference?

File Systems

- What are the two abstractions of a filesystem, and why does this dichotomy exist?
- What are the two names every file has? What translates between them?
- Why is a directory considered just a file? What do its data blocks contain?
- How do direct, indirect, double-indirect, and triple-indirect pointers extend the maximum file size?
- Walk through the I/O steps to open("/a/b/c/file.csv"). How many reads on a cold cache?
- Why are inode and directory-entry caches (dcache) so important for performance?
- Name the key system calls for file handling and state what each does (open, read, write, lseek, fsync, close).
- Where does the file offset live — in the inode or the open-file table entry? Why does it matter?

Data Models

- How is a database different from just a bunch of files?
- What are the two levels of a database, and why the dichotomy?
  - Logical Vs Physical
- Name three forms of structured, two of semistructured, and two of unstructured data models.
- Describe two differences between a relation and a DataFrame. Can you store one as the other?
- Can you store a tensor as a relation? Vice versa?
- What is a data lake, and how does its coupling of storage and processing differ from an RDBMS?

Multiprocessing

Parallelism & Task Graphs

- Distinguish concurrency from parallelism. Give an example of each, and one case that is both.
- What are the three Vs that characterize "Big Data"? Why does sampling not always suffice for ML?
- Define a dataflow graph. How does it differ from a coarse-grained task graph (DAG)?
- In task parallelism, what is replication and why is it used? State one pro and one con.
- Define the degree of parallelism of a task graph. Why are extra workers beyond it wasteful?
- Why does a worker's notion extend below the node level (cores/threads)? Give the Dask example.

Speedup & Scaleup

- Define speedup. Given n workers, can you always achieve a speedup of n? Why or why not?
- Distinguish speedup from scaleup. What does linear scaleup mean?
- List reasons speedup may be sublinear.
- Define efficiency in terms of speedup and P. What efficiency corresponds to ideal speedup?
- Why is a task-parallel workload's completion time lower-bounded by the longest path in the task graph?
- How can a scheduler use idle-time knowledge to cut cloud costs?

SIMD, Vectorization and GPUs

SIMD & Vectorization

- Define SIMD. Where does it sit in Flynn's taxonomy (SISD, SIMD, MISD, MIMD)?
- Why can pure Python for-loops not exploit SIMD, while NumPy can? What does NumPy rely on under the hood?
- Explain the scalar vs. vectorized benchmark: why is vectorized addition orders of magnitude faster?
- Define speedup for multi-core parallelism. Why is FLOPs a useful surrogate for completion time?
- Distinguish concurrency from parallelism in the context of SIMD vs. multi-threading.

Amdahl & Gustafson

- State Amdahl’s Law. Define p, (1−p), and n in the speedup formula.
- What is the maximum speedup as $n \to \infty$ for p = 0.5, 0.9, 0.95? Why does the serial fraction cap it?
- Why is "even infinite processors can’t overcome the serial fraction" the key insight of Amdahl’s Law?
- What fixed assumption does Amdahl make about problem size, and why does it break for scaled workloads?
- State Gustafson’s Law (scaled speedup = $n - s(n-1)$). How does it differ in what it holds fixed?
- Give a real example where problem size and core count grow together, making Amdahl too pessimistic.
- As data grows, why does the parallel fraction p tend toward 1?

GPU Architecture & Memory

- Contrast the design philosophy of CPUs (latency-optimized) vs. GPUs (throughput-optimized) in terms of cores, caches, and control.
- Order the GPU memory hierarchy (registers, shared/L1, L2, global HBM, system RAM via PCIe) by latency. Why does locality matter so much on a GPU?
- Explain the CUDA execution model: grid, block, thread. What is a warp, and how many threads does it contain?
- What do threads within a block share, and why is that significant for performance?
- Why does matrix multiply sit near the ridge point, making it GPU-efficient? When does adding cores NOT help?
- Define arithmetic intensity (FLOPs/byte) and explain its role in the roofline.

GPU Programming in Python

- What is CuPy, and how does its API relate to NumPy? When is it worth moving arrays to the GPU?
- Why minimize host–device transfers in a GPU program?
- In the chained CuPy expression cp.exp(A) + cp.sqrt(B), how many kernels are launched, and why does that matter?
- What does Numba's @cuda.jit decorator do? How does a thread compute its global index via cuda.grid(1)?
- Distinguish Data Parallel from Distributed Data Parallel.

Scaleable Data Access

Hierarchy & Paged Access

- List the tiers of the memory hierarchy (cache, DRAM, SSD, HDD) and order them by capacity, bandwidth, latency, and cost.
- Roughly how much slower is HDD bandwidth than DRAM, and how much cheaper per TB? Why does this tradeoff drive system design?
- Why does pandas.read_csv() fail or thrash on files larger than DRAM? What does OS virtual memory do, and why is swapping so costly?
- A NumPy array of 100M float64 values takes how much memory? Why won't a 10 GB CSV fit on most laptops?
- Describe paged access. What is a buffer pool, and how does it bound memory use?
- How does paged/chunked processing enable datasets 10–100x larger than DRAM?

I/O Cost Model

- State the I/O cost model: how is I/O cost defined, and how is latency computed from it?
- For a 40 GB file with 4 KB pages, how many pages are read? Compute scan time on HDD (200 MB/s), SSD (3 GB/s), and S3 (1 GB/s).
- Why is the HDD scan ~15x slower than SSD, and the S3 read ~3x slower? What dominates each case?
- Why can random I/O on an HDD be 100–1000x slower than sequential? What physical costs cause this?
- A 10M-page random read takes hours while a sequential read takes ~200s — explain the gap and its implication for layout choice.
- Why is reducing the number of page reads described as the #1 optimization lever for large datasets?

Storage Layouts

- Describe row-store, column-store, and tiled (PAX) layouts. Give a representative file format or system for each.
- Using the 6x4 example (page = 4 cells), give the I/O cost of Q1 (SELECT *), Q2 (SUM(B)), and Q3 (2 rows) under row-store.
- Why does column-store cost 8 pages for a full scan but only 2 for SUM(B)? Why does a 2-row lookup cost 4 instead of 2?
- Why is the tiled layout a "middle ground"? Give its I/O cost for the three benchmark queries.
- For each workload (full scan, single-column aggregate, row lookup, ML row sampling, matrix multiply), which layout wins and why?
- State the key principle: why is there no universally best layout?
- Why does column-store compress better than row-store, and why is appending to it expensive?

Cache Replacement

- What question does a cache replacement policy answer? Why does it only matter when the buffer pool is full?
- For which access patterns is LRU good, and for which is it bad? Why does LRU perform poorly on a filescan?
- When does MRU win over LRU? Explain why MRU is optimal for a repeated sequential scan that exceeds the buffer.
- In the 4-frame, 5-page example, why does the repeated pattern P1..P4,P1..P4,P5 give 5 misses (optimal) under LRU?
- Why does the cyclic filescan pattern give 10 misses (worst case) under 4-frame LRU? Why would MRU achieve 5?

Scaling Data Science operations

DB Operations & I/O Cost

1. State the recurring pattern behind every scalable DS operation. What two quantities make up its I/O cost?
2. SELECT C FROM R on a 6-page, 4-column table: give the page reads for row-store vs col-store and explain the difference.
3. Why does a non-deduplicating projection keep duplicates, and why does this not change the DRAM footprint?
4. For SELECT MAX(A) FROM R, what is the DRAM footprint and why is it O(1) regardless of dataset size?
5. Which simple aggregates (besides MAX) use a single running scalar? Why does AVG need a little more state?
6. For SELECT A, SUM(D) FROM R GROUP BY A, what data structure lives in DRAM and what determines its size?
7. Define thrashing in the GROUP BY context. When does the hash table exceed DRAM?
8. Explain the divide-and-conquer fix for GROUP BY overflow. What do the two 'tricks' (hash-partition, reduce-early) accomplish?
9. For a relational select with a WHERE predicate, why is LRU a safe eviction policy during a filescan?
10. Why is row-store an acceptable layout for SELECT * WHERE B = '3b' but a poor one for SELECT C FROM R?

Matrix Algebra & Gradient Descent

11. Define the squared Frobenius norm. Why does computing it reduce to the same pattern as a SUM aggregate?
12. For a full matrix sum, does a tiled layout reduce I/O versus row-store? Justify your answer.
13. Why is row-store a poor fit for matrix multiply and the Gram matrix, while tiled layout 'scales on both dimensions'?
14. Where does the Gram matrix M^T M appear in ML? Name at least two settings.
15. For 6x4 M in 2x2 tiles, the walkthrough gives 18 reads + 4 writes. Why do off-diagonal output tiles cost more reads than diagonal ones?
16. In the tiled Gram computation, why are never more than 2 tiles needed in DRAM at once?
17. Write the ERM objective. Why is gradient descent preferred over the O(d^3) closed-form solution?
18. Why is the BGD gradient analogous to a SQL SUM over a column? What stays in DRAM across page reads?
19. State the I/O cost of one BGD epoch and explain why monitoring loss each epoch can double it.
20. Why must w stay DRAM-resident, and why is BGD access pattern purely sequential (no random access)?

SGD, Tradeoffs & Synthesis

21. State the two main drawbacks of Batch GD that motivate SGD.
22. How does SGD approximate the gradient? Why is the estimate 'noisy but directionally correct'?
23. With n = 1M examples and B = 100, how many weight updates per epoch under SGD vs BGD?
24. Why does mini-batch noise help with non-convex (deep learning) loss surfaces?
25. List the SGD access-pattern steps from raw dataset D to mini-batch updates. Where does the shuffle fit?
26. Why does on-disk shuffling require external merge sort, and what is its approximate cost relative to a filescan?
27. State the precise tradeoff of shuffle-once-upfront vs shuffle-every-epoch over K epochs.
28. Compare BGD and SGD on: updates/epoch, gradient quality, I/O per update, I/O per epoch, convergence speed.
29. Worked: a 100 GB Parquet file has 20 equal-width columns; you sum over 4. What is the I/O cost, and why?
30. Synthesis: across all operations, what single recipe makes them scalable to any dataset size, and what does layout choice determine?

BSP + Hadoop + Spark

BSP & Parallelism

- Distinguish SIMD (data parallelism), task/dataflow parallelism, and BSP. Give one real example of each from data-science practice.
- Define a BSP superstep. Name its three phases in order and state what happens in each.
- What does the barrier 'buy' (list at least two guarantees) and what does it 'cost' (list at least two)?
- Using the shared-counter word-count example, explain how removing the barrier produces a race condition. Why can the same code give 7 on one run and 6 on another?
- Why is BSP a good fit for iterative graph algorithms like PageRank? Map one PageRank iteration onto the compute / communicate / barrier phases.
- Give the rough BSP per-superstep cost model. Why does a single straggler set the pace of the whole job?
- State two workloads where BSP is a poor fit, and name an alternative model that can outperform it for ML training.

MapReduce, Hadoop & HDFS

- Give the type signatures of map and reduce. Why must map be stateless, and what does that enable?
- Trace word count over the inputs 'the cat sat' and 'the dog ran' through map, shuffle (group by key), and reduce.
- What does the shuffle phase do mechanically (partition, sort, transfer)? Why is it usually the bottleneck rather than the map or reduce code?
- What is a combiner and how does it reduce shuffle cost? For word count, what would the combiner compute?
- List two things MapReduce 'nailed' and two things that 'hurt' — and explain why iterative algorithms are painful in MapReduce.
- Describe the roles of the NameNode vs DataNodes in HDFS. Why is the NameNode kept out of the data path?
- HDFS default block size is ~128 MB with 3x replication. Why does HDFS favor a few large files over many small ones, and append-only over random updates?
- What problem does YARN solve, and how does decoupling resource management from execution let Spark, Tez, and others share one cluster?

Spark & RDDs

- Name the three MapReduce limitations Spark was designed to fix, and Spark's answer to each.
- Expand RDD. Explain each of the four properties: resilient, distributed, dataset, immutable.
- Distinguish transformations from actions. Classify each: map, filter, reduceByKey, collect, count, join, saveAsTextFile.
- What is lazy evaluation, and why does Spark wait for an action before running anything? What is the benefit?
- Define lineage. When a worker dies, how does Spark recover lost partitions without replicating data, and what is the trade-off for long lineages?
- What does checkpoint() do, and when would you use it in an iterative job?
- Sketch the Spark architecture: driver, cluster manager, executors. Where is the DAG built and where does cached data live?
- Compare MapReduce and Spark on intermediate data, iterative jobs, API breadth, latency, and fault tolerance. Name one workload best suited to each.

MR in Data Science

MR Fundamentals & SQL Operations

- **The three questions: for any MR job, state how the input is split, what (key, value) Map emits, and what Reduce does per key.**
- Why does a relational SELECT (filter on a predicate) need no Reduce step? What are its disk and network costs, and why?
- Plain PROJECT is Map-only, but SELECT DISTINCT is not. Explain what forces a Reduce step for deduplication.
- Write the Map/Reduce recipe for SELECT MAX(A) FROM R. Why use a single dummy key, and what is the network cost in terms of #workers?
- Classify each aggregate (distributive / algebraic / holistic) and give the per-shard emit size:
  - SUM, COUNT, MIN, MAX
  - AVG, VARIANCE, STDEV
  - MEDIAN, MODE, PERCENTILE
- Why can AVG ship just (sum, count) per shard while MEDIAN cannot be summarized in constant size? Show MEDIAN(whole) is not a function of the shard medians.
- For SELECT A, SUM(D) FROM R GROUP BY A: what becomes the Map key? Estimate the network cost in terms of #groups and #workers.
- A GROUP BY on a high-cardinality key overflows reducer memory. Name three fixes (more reducers, combiner, spill to disk) and explain each.
- What is data skew on a key, and why does one heavy value (e.g. NULL or a viral user) stall the whole job? What does salting do?

Numeric & ML Operations

- Show that summing a tiled matrix is an algebraic aggregate. What does each worker emit, and what is the network cost?
- Write the MR plan for the Lp norm of a matrix. Where is the final p-th root applied, and why can the inner sum be split across shards?
- Batch Gradient Descent: explain why the full-batch gradient is the sum of per-example gradients, making it 'MR-friendly'.
- In BGD, what is broadcast to every worker at the start of each step, and what is the per-epoch network cost in terms of #workers and $|\theta|$?
- Contrast how Hadoop vs. Spark run the per-step BGD loop. Why is Spark much faster for iterative training?
- Give three reasons SGD does NOT fit cleanly into MapReduce:
  - Sequential dependency between mini-batches
  - No constant-size (algebraic) shard summary
  - Update order matters (non-commutative)
- Describe the Parameter Server pattern. What barrier is dropped, what staleness is accepted, and why is SGD robust to it?
- What is the main cost of the Parameter Server approach, and why does it arise?

Advanced Patterns & Hybrid Parallelism

- Why do many algorithms need chained MR jobs? In Hadoop, where is each stage's output stored, and what is the I/O penalty Spark avoids?
- K-Means as two MR jobs per iteration: state what the Assignment job and the Update job each compute, and which one is Map-only.
- Broadcast join vs. reduce-side join: when is each used, which avoids shuffling the big table, and what are the disk/network costs of each?
- In a reduce-side join, why is each row tagged with its source table? Walk through how the reducer produces the joined output for one key.
- Inverted index: what does Map emit per word, and what does Reduce produce? Why was this the original MapReduce use case at Google?
- Compare task parallelism and BSP data parallelism — give one pro and one con of each.
- Scheduling problem: with 4 tasks on a shared dataset, the schedules cost 30 (task-par), 20 (data-par), and 18 (hybrid). Explain why hybrid wins.
- Speedup: a query takes 20 min on 1 node and x min on 5 nodes. Classify as sublinear / linear / superlinear for x = 7, x = 4, x = 3.
- Scaleup: training takes 40 min on 1 node; data is tripled and run on 3 nodes in x min. Compute the speedup vs. 120 min baseline for x = 50 and x = 40.
- For the given task graph (D→T1=20, D→T2=x, T1/T2→T3=y, →T4=10): what is the degree of parallelism, and the max x and y for completion time 30?

ML Ops

The Model-Building Pipeline

- **Name the four stages of the model-building pipeline and state what each one produces. Why is it described as a loop rather than a line?**
- What is a feature? Give an example of turning each raw field into a useful feature: a timestamp, a postal address, a skewed price, and a height/weight pair.
- Why do good features make the relationship the model must learn 'simpler and more linear', and how does this encode domain knowledge the algorithm cannot discover itself?
- Contrast filter, wrapper, and embedded feature-selection methods on speed, accuracy, and whether they are model-agnostic.
- Give three reasons to prune features even when more data is available.
- Define the bias-variance tradeoff. For each regime, give the symptom and the fix:
  - High bias (underfit)
  - High variance (overfit)
- State the No Free Lunch theorem in your own words. What practical workflow does it imply (baseline, matching assumptions, empirical comparison)?
- Rank the algorithm families (linear, RF, boosting, SVM/kNN, neural net) by training cost and by how well they parallelize. Why is Random Forest 'embarrassingly parallel'?
- Distinguish a parameter from a hyperparameter. Give two examples of each, and explain why tuning happens before training.
- Why does the validation scheme change with the problem? Contrast standard k-fold, stratified k-fold, and forward-chaining splits, and say when each is required.
- Why can two models with identical accuracy need different evaluation metrics? Pick the right primary metric for fraud detection vs. price regression and justify each.

Training as a Systems Problem

- **Production adds three questions to 'does the model work?' — name them (accuracy, performance, cost) and say what drives each.**
- Contrast training and serving as workloads on: when each runs, its traffic pattern, its key metric (throughput vs. latency), and its cost shape.
- Define a task and a task DAG. What does an edge mean, and what does it mean for two tasks to be independent?
- Define critical path and maximum concurrency. State the two rules: the lower bound on finish time, and the number of workers worth having.
- Why can adding machines never beat the critical path? What must you change instead to finish faster?
- Define throughput and latency at training time. Give one way each is maximized/minimized, and explain why they trade against each other.
- Compare the four compute regimes (single GPU, multi-GPU one node, multi-node, spot). What is 'coordination cost' and why does it kill naive scale-out?
- Contrast data-parallel and model-parallel training: what does each worker hold, when is each needed, and what is the practical escalation sequence?
- Compare FIFO, fair-share, and gang scheduling. Why is gang scheduling required for distributed training? Expand wall_clock = queue_wait + compute + startup + data_load.

The Serving Workload

- **Contrast online and batch inference on latency target, traffic pattern, what drives them, and how each scales.**
- Why do averages lie for latency? Define p50, p95, p99, and 'the tail', and explain why an SLA is set on a percentile. What lives in the tail?
- Describe the three throughput knobs (batching, concurrency, replicas) and their effect on latency. Why tune batching and concurrency before adding replicas?
- Replica sizing: target 1,000 QPS, each replica handles ~60 RPS, with 30% headroom. Show the arithmetic that gives 22 replicas. Name two gotchas to budget for.
- Why is hardware chosen on cost-per-request, not cost-per-machine? Match CPU / GPU / multi-GPU / vendor-API / edge to model classes.
- When does autoscaling pay off and when does it bite? Why scale on rolling p95 latency rather than CPU, and what is the scale-to-zero risk?
- Name four ways to avoid the model call entirely (result cache, precompute, feature store, quantize/distill) and the main risk or benefit of each.

The Data Plane & MLOps

- **Contrast batch and streaming feature pipelines on freshness, serve-time latency, and cost shape. Why are most real systems hybrid?**
- What is train/serve skew and why is it 'the most expensive silent bug'? Give two mitigations.
- What does an orchestrator (Airflow/Kubeflow) do for a training DAG? List at least four responsibilities.
- List the six stages of the closed MLOps loop, and the four things that keep it closed (automation, lineage, triggers, rollback).
- What four ingredients make a run reproducible? Why is 'the same query last Tuesday' not data pinning and 'it worked on my laptop' not an environment?
- Describe the model-registry promotion flow (train -> validate -> canary -> monitor -> promote). What is a canary and why route only 5% of traffic to it?
- Name the four things to monitor in production (input drift, prediction drift, model quality, system metrics). Why do drift signals arrive before quality signals?
- Compare scheduled, drift-triggered, and performance-triggered retraining on responsiveness and cost. Why is a hybrid trigger typical?
- Why is compute rarely the biggest annual TCO line, and which line usually is? Name three levers that actually move the bill.
- For small classical models, large in-house neural nets, and LLM-via-API: contrast training cost, serving hardware, latency profile, and MLOps burden. Why is the choice rarely about accuracy alone?